

Basics in Applied Mathematics | Winter term 2026/27 | University of Freiburg

Part I: Numerics

Lecture notes

Prof. Dr. Diyora Salimova

Department of Applied Mathematics, University of Freiburg

Course homepage:

https://pfaffelh.github.io/hp/2026WS_BAM.html

Version of October 9, 2026.

Contents

Preface	ii
1 Introduction, floating-point arithmetic, conditioning and norms	1
1.1 What this part of the course is about	1
1.2 Floating-point numbers	2
1.2.1 Representation	2
1.2.2 Rounding and the standard model	3
1.2.3 Cancellation	4
1.3 Condition of a problem and stability of an algorithm	4
1.3.1 Condition numbers	4
1.3.2 Stability of an algorithm	5
1.3.3 Cost of an algorithm	6
1.4 Vector and matrix norms	6
1.4.1 Vector norms	6
1.4.2 Matrix norms	7
1.5 Summary	9
2 Linear systems: condition number, Gaussian elimination, LU and Cholesky	10
2.1 The problem	10
2.2 Conditioning of linear systems	10
2.2.1 Perturbation of the right-hand side	10
2.2.2 Perturbation of the matrix	11
2.2.3 Properties and examples	11
2.3 Triangular systems	12
2.4 Gaussian elimination and the LU decomposition	12
2.4.1 The algorithm	12
2.4.2 Solving $Ax = b$ with the LU decomposition	13
2.4.3 Pivoting	14
2.5 Symmetric positive definite matrices and the Cholesky decomposition	14
2.6 Exploiting structure: banded and sparse matrices	16
2.7 Summary	16

Preface

These notes cover the numerics part of *Basics in Applied Mathematics*: five weeks with two lectures each and a weekly tutorial.

This material was prepared with the help of AI. The selection and structure of the material and every final formulation are mine, as is the responsibility for any remaining errors; please report them to me.

Literature

- S. Bartels, *Numerical Mathematics 3×9*, Springer 2025 – the reference text for this part.
- A. Quarteroni, R. Sacco, F. Saleri, *Numerical Mathematics*, Springer – broad and detailed.
- L. N. Trefethen, D. Bau, *Numerical Linear Algebra*, SIAM – Lectures 3–6; an elegant treatment of QR, SVD and conditioning.
- E. Süli, D. Mayers, *An Introduction to Numerical Analysis*, Cambridge University Press – very accessible, with full proofs.
- P. Deuffhard, A. Hohmann, *Numerical Analysis in Modern Scientific Computing*, Springer – condition and stability.
- E. Hairer, S. P. Nørsett, G. Wanner, *Solving Ordinary Differential Equations I*, and E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, Springer – for Lecture 10.

1 Introduction, floating-point arithmetic, conditioning and norms

Learning goals. After this lecture you can

- (i) name the four sources of error in a numerical computation,
- (ii) explain how real numbers are stored in a computer and why $0.1 + 0.2 \neq 0.3$,
- (iii) tell a *well-conditioned problem* from a *stable algorithm*,
- (iv) measure the size of vectors and matrices with norms and compute $\|A\|_1$, $\|A\|_\infty$, $\|A\|_2$.

1.1 What this part of the course is about

Numerical mathematics designs and analyses **algorithms that compute approximate solutions of mathematical problems** using a finite number of arithmetic operations on numbers of finite precision. The typical problems in this part are

- solve a linear system $Ax = b$ (Chapters 2, 6), possibly with more equations than unknowns (Chapter 3),
- compute eigenvalues and singular values of a matrix (Chapters 4, 5),
- solve a nonlinear equation $f(x) = 0$ (Chapter 7),
- reconstruct a function from finitely many values (Chapter 8) and compute an integral (Chapter 9),
- solve an ordinary differential equation (Chapter 10).

For each problem we ask three questions, and they recur in every lecture:

1. **Accuracy.** How far is the computed result from the true solution, and how does the error depend on the parameters of the method (step size, number of iterations)?
2. **Cost.** How many operations and how much memory does the method need as a function of the problem size n ?
3. **Robustness.** Does the method still work when the data are perturbed, e.g. by measurement errors or by rounding in the computer?

Four sources of error. In any computation with a mathematical model we have to distinguish

- the **model error** (the equations do not describe reality exactly),
- the **data error** (the input is only known approximately),
- the **method error** (we replace the exact mathematical problem by a computable approximation: a series by a finite sum, an integral by a finite sum, a limit by an iterate),
- the **rounding error** (the computer stores numbers with finite precision).

Numerical analysis is concerned with the last two, and with how the algorithm reacts to the second. The first example shows that two mathematically identical formulas can behave very differently in a computer.

Example 1.1 (two formulas, one function). For $x > 0$ the functions

$$f(x) = \frac{1}{x} - \frac{1}{x+1}, \quad g(x) = \frac{1}{x(x+1)}$$

are equal. Let us evaluate both for $x_k = 10^k$ and compare the relative difference $|f - g|/|g|$.

k	x	f(x)	g(x)	rel. difference
1	1e+01	9.0909090909e-03	9.0909090909e-03	3.82e-16
2	1e+02	9.9009900990e-05	9.9009900990e-05	8.21e-16
3	1e+03	9.9900099900e-07	9.9900099900e-07	2.16e-14
4	1e+04	9.9990001000e-09	9.9990001000e-09	6.39e-14
5	1e+05	9.9999000010e-11	9.9999000010e-11	6.25e-13
6	1e+06	9.9999899989e-13	9.9999900000e-13	1.08e-10
7	1e+07	9.9999989907e-15	9.9999990000e-15	9.28e-10
8	1e+08	1.0000000003e-16	9.9999999000e-17	1.03e-08
9	1e+09	1.0000001492e-18	9.9999999900e-19	1.50e-07
10	1e+10	1.0000006145e-20	9.9999999990e-21	6.15e-07
11	1e+11	9.9999996827e-23	9.9999999999e-23	3.17e-08
12	1e+12	1.0000484359e-24	1.0000000000e-24	4.84e-05
13	1e+13	1.0009067165e-26	1.0000000000e-26	9.07e-04
14	1e+14	9.9396474058e-29	1.0000000000e-28	6.04e-03
15	1e+15	9.8607613153e-31	1.0000000000e-30	1.39e-02
16	1e+16	0.0000000000e+00	1.0000000000e-32	1.00e+00

For $x = 10^k$ the formula f loses about k of its 16 significant digits. The reason — the two reciprocals agree in their first k digits and the subtraction cancels exactly those. For $x = 10^{16}$ it even returns 0, because $x + 1$ rounds to x — while g is fine. Nothing is wrong with f mathematically; the problem is the *subtraction of two nearly equal numbers* in floating-point arithmetic, called **cancellation**. Understanding this is the first goal of the lecture.

1.2 Floating-point numbers

1.2.1 Representation

A computer cannot store the real number π , only finitely many digits of it. The standard way of representing real numbers is the *floating-point* format.

Definition 1.2. Let $\beta \geq 2$ (the *base*), $t \geq 1$ (the *precision*) and $e_{\min} \leq e_{\max}$ be integers. The set of **floating-point numbers** is

$$\mathbb{F} = \mathbb{F}(\beta, t, e_{\min}, e_{\max}) = \left\{ \pm \beta^e \sum_{i=1}^t d_i \beta^{-i} : d_i \in \{0, \dots, \beta-1\}, d_1 \neq 0, e_{\min} \leq e \leq e_{\max} \right\} \cup \{0\}.$$

The number $m = \sum_{i=1}^t d_i \beta^{-i} \in [\beta^{-1}, 1)$ is the *mantissa* (or *significand*), e the *exponent*. The condition $d_1 \neq 0$ makes the representation unique (*normalised* numbers). The set defined here consists of the normalised numbers and 0; IEEE arithmetic additionally contains *subnormal* numbers ($d_1 = 0$ with $e = e_{\min}$, which fill the gap between 0 and the smallest normalised number with reduced precision), $\pm\infty$ and NaN. Everything below refers to the normal range.

Virtually all computers use the IEEE 754 standard: **double precision** has $\beta = 2$, $t = 53$, $e_{\min} = -1021$, $e_{\max} = 1024$, stored in 64 bits (1 sign bit, 11 exponent bits, 52 mantissa bits — the leading digit $d_1 = 1$ is not stored). Single precision (**float32**) has $t = 24$. The largest double is about $1.8 \cdot 10^{308}$, the smallest positive normalised one about $2.2 \cdot 10^{-308}$.

Two facts to remember:

- $\mathbb{F}$ is a **finite** set (about $1.8 \cdot 10^{19}$ doubles). In particular it is not closed under $+$, $-$, $\cdot$, $/$.
- The elements of $\mathbb{F}$ are **not equally spaced**: between 1 and 2 there are 2^{52} doubles with spacing 2^{-52} , between 2^k and 2^{k+1} the spacing is 2^{k-52} . The *relative* spacing lies between 2^{-53} and 2^{-52} , independently of the magnitude.

```
spacing of doubles near 1e+00: 2.220e-16 (relative: 2.220e-16)
spacing of doubles near 1e+03: 1.137e-13 (relative: 1.137e-16)
spacing of doubles near 1e+06: 1.164e-10 (relative: 1.164e-16)
spacing of doubles near 1e+15: 1.250e-01 (relative: 1.250e-16)
0.1 + 0.2 == 0.3 ? False    0.1+0.2 = 0.30000000000000004
```

The last line is not a bug: 0.1 is not a double (in base 2 it has an infinite periodic expansion), so 0.1 is already the nearest double to 0.1, and the sum of two rounded numbers, rounded again, need not equal the rounded 0.3.

1.2.2 Rounding and the standard model

Definition 1.3. The **rounding** $\text{fl}: \mathbb{R} \rightarrow \mathbb{F}$ maps x to a floating-point number nearest to x (with a tie-breaking rule). The **machine precision** (unit roundoff) is

$$u = \frac{1}{2}\beta^{1-t}, \quad \text{for doubles: } u = 2^{-53} \approx 1.11 \cdot 10^{-16}.$$

The number $\varepsilon_{\text{mach}} = \beta^{1-t} = 2u = 2^{-52} \approx 2.22 \cdot 10^{-16}$ is the distance from 1 to the next larger floating-point number (`np.finfo(float).eps`).

Theorem 1.4 (rounding error). *Let $x \neq 0$ be a real number that is neither too small nor too large for $\mathbb{F}$:*

$$\beta^{e_{\min}-1} \leq |x| \leq x_{\max} := (1 - \beta^{-t}) \beta^{e_{\max}}.$$

(The left bound is the smallest positive normalised number, the right bound the largest floating-point number; the two conditions are called no underflow and no overflow.) Then rounding changes x by a relative error of at most u :

$$\frac{|\text{fl}(x) - x|}{|x|} \leq u = \frac{1}{2}\beta^{1-t}.$$

Equivalently: $\text{fl}(x) = x(1 + \delta)$ for some number δ (depending on x) with $|\delta| \leq u$.

Proof. Step 1: locate x . There is exactly one integer e with $\beta^{e-1} \leq |x| < \beta^e$, and the hypothesis on $|x|$ gives $e_{\min} \leq e \leq e_{\max}$. (In the notation of Definition 1.2, $x = \pm\beta^e m$ with the real mantissa $m = |x|/\beta^e \in [\beta^{-1}, 1)$.)

Step 2: the grid. The floating-point numbers in $[\beta^{e-1}, \beta^e]$ are the numbers $\beta^e \cdot 0.d_1 d_2 \dots d_t = k \beta^{e-t}$ with $k = \beta^{t-1}, \dots, \beta^t$: an equally spaced grid with spacing β^{e-t} . Rounding to nearest picks the closest grid point, which is at most half a spacing away:

$$|\text{fl}(x) - x| \leq \frac{1}{2}\beta^{e-t}.$$

Step 3: divide by $|x|$. Since $|x| \geq \beta^{e-1}$, we have

$$\frac{|\text{fl}(x) - x|}{|x|} \leq \frac{\frac{1}{2}\beta^{e-t}}{\beta^{e-1}} = \frac{1}{2}\beta^{1-t} = u.$$

The exponent e has cancelled: the bound is the same for every magnitude of x . For the second form set $\delta := (\text{fl}(x) - x)/x$; then $\text{fl}(x) = x(1 + \delta)$ and $|\delta| \leq u$ by the first form. $\square$

The bound is sharp. In Step 3 the worst case is x just above β^{e-1} , at the bottom of the grid interval, where the spacing is largest relative to x ; at the top, just below β^e , the relative error is β times smaller. For doubles, $x = 1 + 2^{-53}$ lies halfway between its neighbours 1 and $1 + 2^{-52}$, is rounded to 1, and has relative error $2^{-53}/(1 + 2^{-53}) \approx u$. This also explains the factor 2 between $u = 2^{-53}$ and $\varepsilon_{\text{mach}} = 2^{-52}$: $\varepsilon_{\text{mach}}$ is the spacing of the doubles at 1, and the rounding error is at most half of the spacing.

The IEEE standard guarantees that the four basic operations are *correctly rounded*: the computed result of $a \circ b$, $\circ \in \{+, -, \cdot, /\}$, is $\text{fl}(a \circ b)$. This gives the **standard model of floating-point arithmetic**:

$$\boxed{\text{fl}(a \circ b) = (a \circ b)(1 + \delta), \quad |\delta| \leq u}$$

for all $a, b \in \mathbb{F}$ — under the assumptions behind Theorem 1.4: rounding to nearest (the IEEE default), $b \neq 0$ for the division, and the exact result $a \circ b$ in the normal range (overflow and underflow excluded). We assume this throughout. Each single operation is thus accurate to a relative error u . The difficulty is what happens when many operations are combined.

```
machine epsilon: 2.220446049250313e-16 = 2^-52 = 2.220446049250313e-16
smallest power of two e with 1+e > 1 : 2.220446049250313e-16
floating point addition is not associative: (1 + 1e-16) - 1 = 0.0 , 1 + (1e-16 - 1) =
1.1102230246251565e-16
```

1.2.3 Cancellation

Consider the subtraction of two numbers that are themselves the result of earlier computations, $\tilde{a} = a(1 + \delta_a)$, $\tilde{b} = b(1 + \delta_b)$ with $|\delta_a|, |\delta_b| \leq u$. Even if the subtraction itself is exact, the relative error of the result is

$$\frac{|(\tilde{a} - \tilde{b}) - (a - b)|}{|a - b|} = \frac{|a\delta_a - b\delta_b|}{|a - b|} \leq u \frac{|a| + |b|}{|a - b|}.$$

If $a \approx b$ the factor $(|a| + |b|)/|a - b|$ is huge: the *relative* error of the input is amplified. This is **cancellation**: the leading digits of a and b agree and cancel, leaving the trailing digits — which carry the rounding errors — as the significant digits of the result. In Example 1.1, $1/x$ and $1/(x + 1)$ agree in the first k digits for $x = 10^k$ (their difference is 10^{-k} times their size), hence roughly k of the 16 digits are lost.

Example 1.5 (quadratic equation). The roots of $x^2 + px + q = 0$ are $x_{1,2} = -\frac{p}{2} \pm \sqrt{p^2/4 - q}$. If $p^2 \gg |q|$ then $\sqrt{p^2/4 - q} \approx |p|/2$ and one of the two formulas subtracts nearly equal numbers. The remedy is to compute the root of *larger modulus* by the formula without cancellation and the other from Vieta's rule $x_1 x_2 = q$. For $p = 10^8$ and $q = 1$, we obtain

```
naive : x1 = -7.4505805969e-09   x2 = -1.0000000000e+08
stable: x1 = -1.0000000000e+08   x2 = -1.0000000000e-08
```

The naive formula returns $-7.45 \cdot 10^{-9}$ for the small root — an error of 25 %, after only three operations — while the stable one returns -10^{-8} correct to all digits. **Rule of thumb**: avoid subtracting nearly equal quantities; if a formula does so, look for an algebraically equivalent one that does not.

1.3 Condition of a problem and stability of an algorithm

1.3.1 Condition numbers

Before we blame an algorithm for an inaccurate result, we should ask whether the *problem itself* is sensitive to perturbations of the data. This is a property of the problem, independent of any

algorithm.

Definition 1.6. Let $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ describe a problem (input x , output $f(x)$). The **absolute** and **relative condition numbers** of f at x are

$$\kappa_{\text{abs}}(x) = \lim_{\varepsilon \rightarrow 0} \sup_{0 < \|\Delta x\| \leq \varepsilon} \frac{\|f(x + \Delta x) - f(x)\|}{\|\Delta x\|},$$

$$\kappa_{\text{rel}}(x) = \lim_{\varepsilon \rightarrow 0} \sup_{0 < \|\Delta x\| \leq \varepsilon} \frac{\|f(x + \Delta x) - f(x)\| / \|f(x)\|}{\|\Delta x\| / \|x\|}$$

(for the relative condition number we assume that $x \neq 0$ and $f(x) \neq 0$). A problem is **well-conditioned** if κ_{rel} is small (say $\lesssim 10^2$) and **ill-conditioned** if it is large. (The norms $\|\cdot\|$ are defined in Section 1.4; in one dimension they are absolute values.)

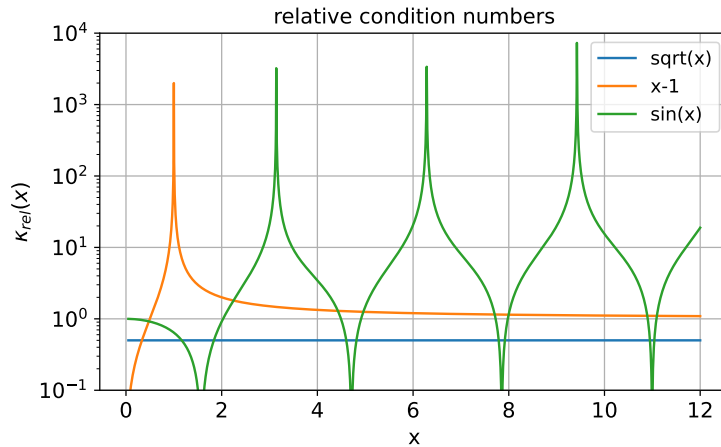
Note that for differentiable $f: \mathbb{R} \rightarrow \mathbb{R}$ the limit gives

$$\kappa_{\text{abs}}(x) = |f'(x)|, \quad \kappa_{\text{rel}}(x) = \frac{|f'(x)| |x|}{|f(x)|}.$$

Example 1.7.

- $f(x) = \sqrt{x}$: $\kappa_{\text{rel}} = \frac{x}{2\sqrt{x}\sqrt{x}} = \frac{1}{2}$. Hence, it is well-conditioned for every $x > 0$.
- $f(x) = x - 1$ (subtraction of a constant): $\kappa_{\text{rel}} = \frac{|x|}{|x-1|} \rightarrow \infty$ as $x \rightarrow 1$. This is cancellation seen as an ill-conditioned problem.
- $f(x) = e^x$: $\kappa_{\text{rel}} = |x|$ — ill-conditioned for large $|x|$.
- $f(x) = \sin x$: $\kappa_{\text{rel}} = |x \cot x|$, huge near the roots $x = k\pi$, $k \neq 0$: a small change of x near 1000π changes $\sin x$ by a lot relative to its tiny value.

Interpretation: κ_{rel} is a *worst-case* amplification factor. If the input carries a relative error of up to ε (up to u just from storing it), the output error can be as large as about $\kappa_{\text{rel}} \varepsilon$, and *no algorithm can guarantee better*, because the algorithm only ever sees the perturbed input. With $\kappa = 10^k$ we must expect to lose k significant digits — a particular perturbation may of course be harmless, and for some inputs the actual error is much smaller than the bound.



1.3.2 Stability of an algorithm

An **algorithm** for the problem f is a finite sequence of elementary operations that maps the input x to a computed result $\tilde{f}(x)$; in floating-point arithmetic each operation is subject to the standard model. The algorithm is called

- **forward stable** if the relative error $\|\tilde{f}(x) - f(x)\|/\|f(x)\|$ is of the size $\kappa_{\text{rel}}(x) \cdot u$ (i.e. as accurate as the conditioning of the problem allows),
- **backward stable** if the computed result is the *exact* result for a slightly perturbed input: $\tilde{f}(x) = f(\tilde{x})$ with $\|\tilde{x} - x\|/\|x\| = O(u)$.

Backward stability is the stronger property; it implies forward stability, because $\|f(\tilde{x}) - f(x)\|/\|f(x)\| \lesssim \kappa_{\text{rel}} \|\tilde{x} - x\|/\|x\|$. The picture to keep in mind:

$$\text{error of the result} \lesssim \underbrace{\text{condition of the problem}}_{\text{cannot be influenced}} \times \underbrace{\text{error of the algorithm}}_{\text{choose a good algorithm!}}$$

Example 1.8. The problem “compute $f(x) = 1/x - 1/(x+1)$ ” is well-conditioned: $\kappa_{\text{rel}} = |xf'(x)/f(x)| = \frac{2x+1}{x+1} \leq 2$. The algorithm *evaluate* $1/x$, *evaluate* $1/(x+1)$, *subtract* is **unstable** for large x (it introduces cancellation that the problem does not have), the algorithm *evaluate* $1/(x(x+1))$ is stable. This is precisely what Example 1.1 shows.

The recurring lesson of this course: *the error of a computation is a product of the condition of the problem and the stability of the algorithm*. We will compute condition numbers for linear systems (Chapter 2), least squares problems (Chapter 3), eigenvalues (Chapter 5) and root finding (Chapter 7), and design algorithms that are backward stable.

1.3.3 Cost of an algorithm

The second quality criterion is cost. We count **floating-point operations (flops)** — additions, multiplications, etc. — as a function of the size n and are interested in the asymptotic behaviour.

Definition 1.9 (Landau symbols). For sequences $a_n, b_n > 0$ we write $a_n = O(b_n)$ if there are $C > 0$ and n_0 with $a_n \leq C b_n$ for all $n \geq n_0$, and $a_n = o(b_n)$ if $a_n/b_n \rightarrow 0$.

Examples: a scalar product of two vectors in $\mathbb{R}^n$ costs $2n - 1 = O(n)$ flops; a matrix–vector product Ax , $A \in \mathbb{R}^{n \times n}$, costs $O(n^2)$; a matrix–matrix product AB costs $2n^3 - n^2 = O(n^3)$. In the next lecture we will see that solving $Ax = b$ costs $\frac{2}{3}n^3 + O(n^2)$ flops. .

n	A@x [s]	A@B [s]	ratio A@B
500	0.00004	0.00224	
1000	0.00023	0.02027	9.0
2000	0.00121	0.15620	7.7

doubling n multiplies the $O(n^3)$ cost of A@B by about 8 (cache effects and multithreading blur the picture)

1.4 Vector and matrix norms

To measure errors of vectors and matrices, and to define condition numbers for matrix problems, we need norms.

1.4.1 Vector norms

Definition 1.10. A **norm** on $\mathbb{R}^n$ is a map $\|\cdot\| : \mathbb{R}^n \rightarrow [0, \infty)$ with

1. $\|x\| = 0 \iff x = 0$ (definiteness),
2. $\|\alpha x\| = |\alpha| \|x\|$ for $\alpha \in \mathbb{R}$ (homogeneity),
3. $\|x + y\| \leq \|x\| + \|y\|$ (triangle inequality).

The most important examples are the **p -norms**

$$\|x\|_p = \left(\sum_{j=1}^n |x_j|^p \right)^{1/p} \quad (1 \leq p < \infty), \quad \|x\|_\infty = \max_j |x_j|,$$

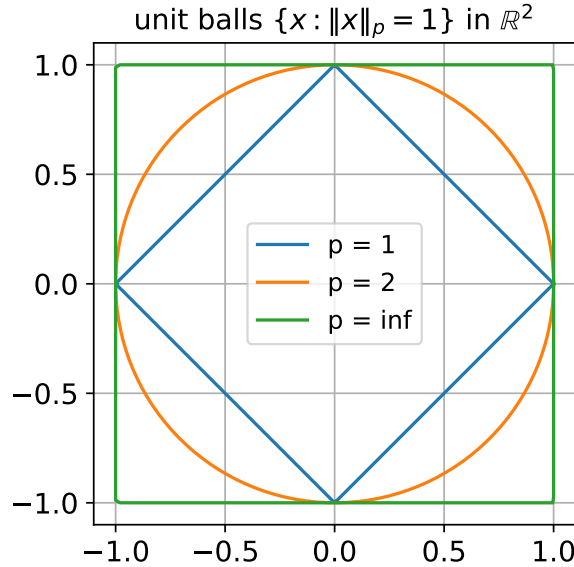
in particular the Euclidean norm $\|x\|_2 = \sqrt{x^\top x}$, the 1-norm (sum of absolute values) and the maximum norm. (That $\|\cdot\|_p$ satisfies the triangle inequality is Minkowski's inequality; for $p = 1, 2, \infty$ it is elementary.)

Theorem 1.11 (equivalence of norms). *For any two norms $\|\cdot\|_a, \|\cdot\|_b$ on $\mathbb{R}^n$ there are constants $0 < c \leq C$ with $c\|x\|_a \leq \|x\|_b \leq C\|x\|_a$ for all $x \in \mathbb{R}^n$. In particular*

$$\begin{aligned} \|x\|_\infty &\leq \|x\|_2 \leq \sqrt{n} \|x\|_\infty, & \|x\|_\infty &\leq \|x\|_1 \leq n \|x\|_\infty, \\ \|x\|_2 &\leq \|x\|_1 \leq \sqrt{n} \|x\|_2. \end{aligned}$$

Proof of the explicit inequalities. $\|x\|_\infty^2 = \max_j |x_j|^2 \leq \sum_j |x_j|^2 \leq n \max_j |x_j|^2$ gives the first pair; the second is analogous. For the third, $\|x\|_2^2 = \sum |x_j|^2 \leq (\sum |x_j|)^2$ and $\|x\|_1 = \sum_j |x_j| \leq \sqrt{n} \|x\|_2$ by the Cauchy–Schwarz inequality. The general statement follows from the compactness of the unit sphere of $\|\cdot\|_\infty$ and the continuity of every norm with respect to $\|\cdot\|_\infty$. $\square$

Consequence: convergence of a sequence of vectors does not depend on the norm used; but the *constants* in error estimates do, and they may depend on n .



1.4.2 Matrix norms

A matrix $A \in \mathbb{R}^{m \times n}$ can be viewed as a vector in $\mathbb{R}^{mn}$, giving e.g. the **Frobenius norm** $\|A\|_F = (\sum_{i,j} a_{ij}^2)^{1/2}$. More useful in the analysis of algorithms are norms that measure *how much a matrix can stretch a vector*.

Definition 1.12. Let $\|\cdot\|$ be a vector norm (on $\mathbb{R}^n$ and $\mathbb{R}^m$). The **induced (operator) norm** of $A \in \mathbb{R}^{m \times n}$ is

$$\|A\| = \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \max_{\|x\|=1} \|Ax\|.$$

The supremum is attained because the unit sphere is compact and $x \mapsto \|Ax\|$ is continuous. By definition,

$$\|Ax\| \leq \|A\| \|x\| \quad \text{for all } x,$$

and $\|A\|$ is the smallest constant with this property.

Proposition 1.13. *Induced norms are norms on $\mathbb{R}^{m \times n}$; moreover $\|I\| = 1$ and they are **submultiplicative**: $\|AB\| \leq \|A\| \|B\|$.*

Proof. Norm axioms are inherited from the vector norm; for submultiplicativity, $\|ABx\| \leq \|A\| \|Bx\| \leq \|A\| \|B\| \|x\|$ for all x . $\square$

(The Frobenius norm is also submultiplicative but not induced by any vector norm, since $\|I\|_F = \sqrt{n}$.)

Theorem 1.14 (explicit formulas). *For $A \in \mathbb{R}^{m \times n}$,*

$$\begin{aligned} \|A\|_1 &= \max_{k=1,\dots,n} \sum_{i=1}^m |a_{ik}| \quad (\text{max. column sum}), \\ \|A\|_\infty &= \max_{i=1,\dots,m} \sum_{k=1}^n |a_{ik}| \quad (\text{max. row sum}), \\ \|A\|_2 &= \sqrt{\lambda_{\max}(A^\top A)}. \end{aligned}$$

Proof. The case $A = 0$ is immediate.

(∞ -norm): For $\|x\|_\infty = 1$ we have $|(Ax)_i| = |\sum_k a_{ik} x_k| \leq \sum_k |a_{ik}|$, so $\|Ax\|_\infty \leq \max_i \sum_k |a_{ik}|$. Let i^* be a maximising row and choose $x_k = \text{sign}(a_{i^*k})$; then $\|x\|_\infty = 1$ and $(Ax)_{i^*} = \sum_k |a_{i^*k}|$, so equality holds.

(1-norm): For $\|x\|_1 = 1$, $\|Ax\|_1 = \sum_i |\sum_k a_{ik} x_k| \leq \sum_k |x_k| \sum_i |a_{ik}| \leq \max_k \sum_i |a_{ik}|$; equality for $x = e_{k^*}$, the unit vector of a maximising column.

(2-norm): $A^\top A$ is symmetric positive semidefinite, so it has an orthonormal eigenbasis $v_1, \dots, v_n$ with eigenvalues $\lambda_1 \geq \dots \geq \lambda_n \geq 0$. For $x = \sum c_j v_j$ with $\|x\|_2^2 = \sum c_j^2 = 1$, $\|Ax\|_2^2 = x^\top A^\top A x = \sum_j \lambda_j c_j^2 \leq \lambda_1$, with equality for $x = v_1$. $\square$

The quantity $\rho(B) = \max\{|\lambda| : \lambda \text{ eigenvalue of } B\}$ is the **spectral radius** of B ; thus $\|A\|_2 = \sqrt{\rho(A^\top A)}$, and for symmetric A simply $\|A\|_2 = \rho(A)$. The 2-norm is the most natural (it is invariant under orthogonal transformations, $\|QA\|_2 = \|A\|_2$ for $Q^\top Q = I$) but the most expensive to compute; $\|A\|_1$ and $\|A\|_\infty$ cost $O(mn)$ and are used for cheap estimates. In Chapter 4 we will see that $\|A\|_2 = \sigma_1$ is the largest *singular value* of A .

Proposition 1.15. *For every induced norm and every square matrix A : $\rho(A) \leq \|A\|$.*

Proof. If $Av = \lambda v$, $v \neq 0$ (possibly complex; use the complex extension of the norm), then $|\lambda| \|v\| = \|Av\| \leq \|A\| \|v\|$. $\square$

Example. For $A = \begin{pmatrix} 1 & 2 \\ -3 & 1 \end{pmatrix}$ the four norms of Theorem 1.14 and the spectral radius $\rho(A) = \sqrt{7}$ (eigenvalues $1 \pm i\sqrt{6}$):

```
||A||_1 = 4.0 (max column sum)
||A||_inf = 4.0 (max row sum)
||A||_2 = 3.192582403567252 = sqrt(lambda_max(A^T A)) = 3.192582403567252
||A||_F = 3.872983346207417
spectral radius rho(A) = 2.6457513110645907 <= every induced norm
```

1.5 Summary

- Floating-point numbers form a finite, non-uniform set; every operation is accurate to a relative error $u \approx 10^{-16}$ (standard model), but **cancellation** in a subtraction of nearly equal numbers can destroy all accuracy.
- The **condition number** measures the worst-case sensitivity of a *problem* to perturbations of its data; for an ill-conditioned problem no algorithm can guarantee an accurate result from uncertain data. **Stability** is the property of an *algorithm* not to introduce additional error amplification.
- The **cost** of an algorithm is measured in flops; we care about the asymptotic order $O(n^p)$.
- **Norms** measure sizes of vectors and matrices; induced matrix norms satisfy $\|Ax\| \leq \|A\| \|x\|$ and $\|AB\| \leq \|A\| \|B\|$; the formulas for $\|A\|_1$, $\|A\|_\infty$, $\|A\|_2$ are used throughout the course.

2 Linear systems: condition number, Gaussian elimination, LU and Cholesky

Learning goals. (i) Bound the effect of data errors on the solution of $Ax = b$ by the condition number $\text{cond}(A)$;
(ii) understand Gaussian elimination as the factorisation $A = LU$ and know its cost $\frac{2}{3}n^3$;
(iii) explain why pivoting is necessary;
(iv) use the Cholesky factorisation for symmetric positive definite matrices;
(v) exploit band structure.

2.1 The problem

Given $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$, find $x \in \mathbb{R}^n$ with $Ax = b$. Linear systems are the workhorse of scientific computing: discretised differential equations, least squares fits, Newton's method for nonlinear systems (Chapter 7), implicit time stepping (Chapter 10) all lead to linear systems, often with n in the millions.

Recall (linear algebra). For $A \in \mathbb{R}^{n \times n}$ the following are equivalent:

- (a) $Ax = b$ has a unique solution for every b ;
- (b) $\ker A = \{0\}$;
- (c) $\text{rank } A = n$, i.e. the columns of A are linearly independent;
- (d) $\det A \neq 0$;
- (e) 0 is not an eigenvalue of A .

Such A is called *regular* (also: nonsingular or invertible). Cramer's rule $x_i = \det A_i / \det A$ is a formula, not an algorithm: evaluating determinants by expansion costs $O(n!)$ operations.

2.2 Conditioning of linear systems

2.2.1 Perturbation of the right-hand side

Suppose the data b are perturbed to $b + \Delta b$ (measurement error, rounding). The solution changes from x to $x + \Delta x$ with $A\Delta x = \Delta b$. How large is Δx relative to x ?

Definition 2.1. For a regular matrix A and an induced matrix norm $\|\cdot\|$ the **condition number** of A is

$$\text{cond}(A) = \|A\| \|A^{-1}\|.$$

We write $\text{cond}_p(A)$ when the p -norm is used.

Theorem 2.2. Let A be regular, $Ax = b \neq 0$ and $A(x + \Delta x) = b + \Delta b$. Then

$$\frac{\|\Delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\Delta b\|}{\|b\|},$$

and the bound is sharp: for every A there are $b, \Delta b$ for which equality holds.

Proof. From $\Delta x = A^{-1}\Delta b$ we get $\|\Delta x\| \leq \|A^{-1}\| \|\Delta b\|$; from $b = Ax$ we get $\|b\| \leq \|A\| \|x\|$, i.e. $1/\|x\| \leq \|A\|/\|b\|$. Multiplying the two inequalities gives the claim. For sharpness choose x with $\|Ax\| = \|A\| \|x\|$ and Δb with $\|A^{-1}\Delta b\| = \|A^{-1}\| \|\Delta b\|$ (both exist since induced norms are maxima over the unit sphere). $\square$

So $\text{cond}(A)$ is exactly the relative condition number (Definition 1.6) of the problem $b \mapsto A^{-1}b$ in the worst case. Note that $\text{cond}(A) \geq \|AA^{-1}\| = \|I\| = 1$ and $\text{cond}(\alpha A) = \text{cond}(A)$ for $\alpha \neq 0$: the condition number has nothing to do with the *size* of A or of $\det A$.

2.2.2 Perturbation of the matrix

Theorem 2.3. *Let A be regular, ΔA with $\|A^{-1}\| \|\Delta A\| < 1$, $Ax = b \neq 0$ and $(A + \Delta A)(x + \Delta x) = b + \Delta b$. Then $A + \Delta A$ is regular and*

$$\frac{\|\Delta x\|}{\|x\|} \leq \frac{\text{cond}(A)}{1 - \text{cond}(A) \frac{\|\Delta A\|}{\|A\|}} \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta b\|}{\|b\|} \right).$$

Proof. Write $B = A^{-1}\Delta A$, so $\|B\| < 1$. Then $I + B$ is regular (if $(I + B)v = 0$ then $\|v\| = \|Bv\| \leq \|B\| \|v\|$ forces $v = 0$) and $\|(I + B)^{-1}\| \leq 1/(1 - \|B\|)$: from $v = (I + B)^{-1}w$ we get $\|v\| \leq \|w\| + \|B\| \|v\|$. Hence $A + \Delta A = A(I + B)$ is regular. Subtracting $Ax = b$ from the perturbed equation gives $(A + \Delta A)\Delta x = \Delta b - \Delta A x$, so

$$\|\Delta x\| \leq \|(I + B)^{-1}\| \|A^{-1}\| (\|\Delta b\| + \|\Delta A\| \|x\|) \leq \frac{\|A^{-1}\|}{1 - \|A^{-1}\| \|\Delta A\|} (\|\Delta b\| + \|\Delta A\| \|x\|).$$

Dividing by $\|x\|$, using $1/\|x\| \leq \|A\|/\|b\|$ in the first term, and $\|A^{-1}\| \|\Delta A\| = \text{cond}(A) \|\Delta A\|/\|A\|$ gives the claim. $\square$

In words: relative errors in A and b of size ε can produce a relative error of up to about $\text{cond}(A)\varepsilon$ in x (the bound is attained for suitable perturbations). Since the data are stored with relative error $u \approx 10^{-16}$, no algorithm can *guarantee* more than $16 - \log_{10} \text{cond}(A)$ correct digits; for particular data the error may be smaller.

2.2.3 Properties and examples

- For symmetric A with eigenvalues λ_i : $\text{cond}_2(A) = \max |\lambda_i| / \min |\lambda_i|$ (since $\|A\|_2 = \rho(A)$ and $\|A^{-1}\|_2 = \rho(A^{-1}) = 1/\min |\lambda_i|$). In general $\text{cond}_2(A) = \sigma_{\max}/\sigma_{\min}$, the ratio of the largest to the smallest singular value (Chapter 4).
- For orthogonal Q ($Q^T Q = I$): $\text{cond}_2(Q) = 1$. Orthogonal matrices are the best-conditioned matrices; this is why the QR decomposition (Chapter 3) is so useful.
- **Residual versus error.** For a computed $\tilde{x}$ we can evaluate the residual $r = b - A\tilde{x}$ but not the error. The theorem applied with $\Delta b = -r$ gives $\|x - \tilde{x}\|/\|x\| \leq \text{cond}(A) \|r\|/\|b\|$: *a small residual guarantees a small error only for well-conditioned matrices.*

Example 2.5 (Hilbert matrix). $H_n = (1/(i+j-1))_{i,j=1}^n$ arises when fitting polynomials in the monomial basis by least squares. It is symmetric positive definite but extremely ill-conditioned: $\text{cond}_2(H_n)$ grows like $e^{3.5n}$.

n	cond ₂ (H _n)	rel. error of numpy's solve for H _n x = H _n * ones
4	1.55e+04	3.37e-14
6	1.50e+07	4.08e-11
8	1.53e+10	3.71e-08
10	1.60e+13	2.27e-05
12	1.64e+16	1.29e-01
14	3.22e+17	3.02e+00

The error grows in proportion to the condition number (staying below the bound $u \operatorname{cond}_2(H_n)$). From $n = 12$ on, $\operatorname{cond}_2(H_n) \approx 1.6 \cdot 10^{16} > 1/u$: there is no worst-case guarantee of a single correct digit when $u \operatorname{cond}_2$ is of order one, although the residual is tiny (particular right-hand sides can still fare better — the bound is a worst case). This is not the fault of the solver. (For $n = 14$ the true $\operatorname{cond}_2 \approx 1.9 \cdot 10^{19}$; `np.linalg.cond` itself cannot compute it, since it works with rounded singular values.)

2.3 Triangular systems

A matrix $U = (u_{ij})$ is **upper triangular** if $u_{ij} = 0$ for $i > j$, and L is **lower triangular** if $l_{ij} = 0$ for $i < j$. Triangular matrices are regular iff all diagonal entries are nonzero (the determinant is their product). Triangular systems are solved by **substitution**:

$$Ux = b: \quad x_i = \frac{1}{u_{ii}} \left(b_i - \sum_{j=i+1}^n u_{ij} x_j \right), \quad i = n, n-1, \dots, 1 \quad (\text{backward substitution}),$$

$$Ly = b: \quad y_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} y_j \right), \quad i = 1, 2, \dots, n \quad (\text{forward substitution}).$$

Cost: row i needs about $2(n-i)$ flops, in total $\sum_i 2(n-i) \approx n^2$ flops. Substitution is also backward stable (the computed solution solves a system with a triangular matrix perturbed by $O(nu)$ relative to U). A check with a random upper triangular 5×5 matrix (random entries plus the identity) and a random right-hand side gives a residual at rounding level:

```
max residual of backward substitution: 0.0
```

2.4 Gaussian elimination and the LU decomposition

2.4.1 The algorithm

The idea of Gaussian elimination is to transform $Ax = b$ into an equivalent triangular system by subtracting multiples of one equation from the others. Write $A^{(1)} = A$. In step k ($k = 1, \dots, n-1$) we eliminate the unknown x_k from equations $k+1, \dots, n$: with the **pivot** $a_{kk}^{(k)} \neq 0$ and the **multipliers** $l_{ik} = a_{ik}^{(k)} / a_{kk}^{(k)}$ we set

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik} a_{kj}^{(k)}, \quad i, j = k+1, \dots, n,$$

and the same for the right-hand side. After $n-1$ steps $A^{(n)} = U$ is upper triangular.

Observation. Step k is the multiplication from the left with the matrix $L_k = I - \ell_k e_k^\top$, where $\ell_k = (0, \dots, 0, l_{k+1,k}, \dots, l_{nk})^\top$: $A^{(k+1)} = L_k A^{(k)}$. Hence $U = L_{n-1} \cdots L_1 A$, and since

$L_k^{-1} = I + \ell_k e_k^\top$ and $L_1^{-1} \cdots L_{n-1}^{-1} = I + \sum_k \ell_k e_k^\top$ (the products of the cross terms $\ell_k e_k^\top \ell_j e_j^\top$ vanish for $k < j$ because $e_k^\top \ell_j = 0$), we obtain

$$A = LU, \quad L = \begin{pmatrix} 1 & & & \\ \ell_{21} & 1 & & \\ \vdots & & \ddots & \\ \ell_{n1} & \cdots & \ell_{n,n-1} & 1 \end{pmatrix},$$

$$U = \begin{pmatrix} u_{11} & \cdots & u_{1n} \\ & \ddots & \vdots \\ & & u_{nn} \end{pmatrix}.$$

Definition 2.6. A factorisation $A = LU$ with L unit lower triangular and U upper triangular is called an **LU decomposition** of A .

Theorem 2.7 (existence and uniqueness). *A regular matrix $A \in \mathbb{R}^{n \times n}$ has an LU decomposition if and only if all leading principal submatrices $A_k = (a_{ij})_{i,j \leq k}$, $k = 1, \dots, n-1$, are regular. In this case the decomposition is unique.*

Proof. If $A = LU$ then $A_k = (L)_k (U)_k$ for the leading $k \times k$ blocks of L and U , and $\det A_k = \det(U)_k = u_{11} \cdots u_{kk}$. Since $\det A = u_{11} \cdots u_{nn} \neq 0$, all $u_{kk} \neq 0$, so all A_k are regular. Conversely, if all A_k are regular, one shows by induction that the pivots $a_{kk}^{(k)}$ of Gaussian elimination are nonzero: $a_{kk}^{(k)} = \det A_k / \det A_{k-1}$, because the elimination steps $1, \dots, k-1$ act on A_k by unit lower triangular transformations, which do not change $\det A_k$, and turn it into a triangular matrix with diagonal $a_{11}^{(1)}, \dots, a_{kk}^{(k)}$. Uniqueness: if $\tilde{L}\tilde{U} = \hat{L}\hat{U}$ are two LU decompositions, then $\hat{L}^{-1}\tilde{L} = \hat{U}\tilde{U}^{-1}$ is both unit lower and upper triangular, hence $= I$. $\square$

2.4.2 Solving $Ax = b$ with the LU decomposition

$$Ax = b \iff L(Ux) = b : \quad (1) \text{ } Ly = b \text{ (forward substitution),}$$

$$(2) \text{ } Ux = y \text{ (backward substitution).}$$

Cost. Step k of the elimination updates an $(n-k) \times (n-k)$ block with $2(n-k)^2$ flops; in total $\sum_{k=1}^{n-1} 2(n-k)^2 \approx \frac{2}{3}n^3$. The two substitutions cost $2n^2$. Hence

- the decomposition ($\frac{2}{3}n^3$ flops) is the expensive part and is computed **once**;
- each additional right-hand side (e.g. the columns of A^{-1} , or a sequence of time steps with the same matrix) costs only $2n^2$;
- computing A^{-1} explicitly and forming $A^{-1}b$ costs about three times more than solving $Ax = b$ — and is less accurate. **Never invert a matrix to solve a linear system.**
- $\det A = \prod_k u_{kk}$ (up to a sign from pivoting) — this is how determinants are computed in practice.

Example. $A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}$, $b = (5, -2, 9)^\top$. The elimination needs no row exchanges; the

code prints the factors L and U , the check $LU - A$, and the solution $x = (1, 1, 2)^\top$ obtained by forward and backward substitution.

```

L =
[[ 1.  0.  0.]
 [ 2.  1.  0.]
 [-1. -1.  1.]]
U =
[[ 2.  1.  1.]
 [ 0. -8. -2.]
 [ 0.  0.  1.]]
L@U - A =
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
x = [1. 1. 2.]    residual: [0. 0. 0.]

```

2.4.3 Pivoting

Gaussian elimination without pivoting fails for the perfectly harmless regular matrix $A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$ ($a_{11} = 0$), and — more insidiously — it is **unstable** when a pivot is small:

Example 2.8. For $A = \begin{pmatrix} \varepsilon & 1 \\ 1 & 1 \end{pmatrix}$ with $0 < \varepsilon \ll 1$ the multiplier is $l_{21} = 1/\varepsilon$ and $u_{22} = 1 - 1/\varepsilon$. In floating-point arithmetic with $\varepsilon = 10^{-17}$, $\text{fl}(1 - 10^{17}) = -10^{17}$, so the computed factors satisfy $\tilde{L}\tilde{U} = \begin{pmatrix} \varepsilon & 1 \\ 1 & 0 \end{pmatrix}$ — the entry $a_{22} = 1$ has been *completely lost* although $\text{cond}_2(A) \approx 2.6$. The large multiplier amplified the rounding error in u_{22} .

The remedy is to exchange rows so that the pivot is the largest available element: **partial pivoting**. In step k choose $p \geq k$ with $|a_{pk}^{(k)}| = \max_{i \geq k} |a_{ik}^{(k)}|$, swap rows k and p , then eliminate. All multipliers then satisfy $|l_{ik}| \leq 1$. Row swaps are permutation matrices P_k , and one obtains:

Theorem 2.9. For every regular A there are a permutation matrix P , a unit lower triangular L with $|l_{ij}| \leq 1$ and an upper triangular U with

$$PA = LU.$$

The system $Ax = b$ is then solved via $LUx = Pb$. Gaussian elimination with partial pivoting is backward stable in practice: the computed factors satisfy $\tilde{L}\tilde{U} = P(A + \Delta A)$ with $\|\Delta A\|_\infty \leq cn^2 \rho u \|A\|_\infty$, where the growth factor $\rho = \max_{i,j,k} |a_{ij}^{(k)}| / \max_{i,j} |a_{ij}|$ is bounded by 2^{n-1} in theory but is almost always small in practice.

2.5 Symmetric positive definite matrices and the Cholesky decomposition

Definition 2.10. $A \in \mathbb{R}^{n \times n}$ is **symmetric positive definite (SPD)** if $A = A^\top$ and $x^\top Ax > 0$ for all $x \neq 0$.

SPD matrices are ubiquitous: the matrices $A^\top A$ of the normal equations for A with full column rank (Chapter 3), Hessians at non-degenerate minima (at a general minimum the Hessian is only positive *semidefinite*, $x^\top Ax \geq 0$), discretised elliptic operators, covariance matrices of random variables whose centred versions are linearly independent in L^2 (in general only semidefinite).

Proposition 2.11. *Let A be SPD. Then*

- (i) A is regular;
- (ii) all leading principal submatrices A_k are SPD;
- (iii) all eigenvalues of A are positive;
- (iv) $a_{ii} > 0$ and $\max_{ij} |a_{ij}| = \max_i a_{ii}$.

Proof. (i) $Ax = 0 \Rightarrow x^\top Ax = 0 \Rightarrow x = 0$. (ii) For $y \in \mathbb{R}^k \setminus \{0\}$ set $x = (y, 0) \in \mathbb{R}^n$; then $y^\top A_k y = x^\top Ax > 0$. (iii) $Av = \lambda v$ gives $\lambda \|v\|_2^2 = v^\top Av > 0$. (iv) $a_{ii} = e_i^\top A e_i > 0$; and for $i \neq j$ we have $0 < (e_i \pm e_j)^\top A (e_i \pm e_j) = a_{ii} + a_{jj} \pm 2a_{ij}$ gives $|a_{ij}| < \frac{1}{2}(a_{ii} + a_{jj}) \leq \max_i a_{ii}$. $\square$

By (ii) and Theorem 2.7, an SPD matrix has an LU decomposition without pivoting; the symmetry allows us to halve the work.

Theorem 2.12 (Cholesky decomposition). *A is SPD if and only if there is a lower triangular matrix L with positive diagonal entries such that*

$$A = LL^\top.$$

This L is unique.

Proof. “ $\Leftarrow$ ”: $x^\top LL^\top x = \|L^\top x\|_2^2 > 0$ for $x \neq 0$ since $L^\top$ is regular. “ $\Rightarrow$ ”: induction on n ; for $n = 1$, $L = (\sqrt{a_{11}})$. For $n > 1$ write $A = \begin{pmatrix} A_{n-1} & a \\ a^\top & \alpha \end{pmatrix}$. By induction $A_{n-1} = L_{n-1}L_{n-1}^\top$.

We look for $L = \begin{pmatrix} L_{n-1} & 0 \\ \ell^\top & \lambda \end{pmatrix}$; the equation $A = LL^\top$ becomes $L_{n-1}\ell = a$ (solvable by forward substitution) and $\ell^\top \ell + \lambda^2 = \alpha$. We need $\lambda^2 = \alpha - \ell^\top \ell > 0$: this holds because $\det A = \det(A_{n-1})(\alpha - a^\top A_{n-1}^{-1}a)$ (Schur complement) is positive by Prop. 2.11(iii), and $a^\top A_{n-1}^{-1}a = \ell^\top \ell$. Uniqueness follows the same induction. $\square$

Algorithm (column by column). Comparing entries of $A = LL^\top$ gives, for $j = 1, \dots, n$,

$$l_{jj} = \left(a_{jj} - \sum_{k < j} l_{jk}^2 \right)^{1/2},$$

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k < j} l_{ik} l_{jk} \right) \quad (i > j).$$

Properties: cost $\frac{1}{3}n^3$ flops (half of LU), only the lower triangle of A is needed (half the storage), **no pivoting is needed** and the algorithm is backward stable (all $|l_{ij}| \leq \sqrt{a_{ii}}$, since $\sum_{k \leq i} l_{ik}^2 = a_{ii}$). In exact arithmetic a non-positive radicand (a zero or a negative one) occurs if and only if A is not positive definite — the Cholesky algorithm is the cheapest test for positive definiteness. In floating point the test is reliable only for reasonably conditioned matrices: for an SPD matrix with $\text{cond}_2(A) \gtrsim 1/u$ rounding errors alone can produce a non-positive radicand (the computed radicand is the exact one of a slightly perturbed matrix, which may be indefinite).

Example. For the SPD matrix $A = \begin{pmatrix} 4 & 2 & -2 \\ 2 & 10 & 2 \\ -2 & 2 & 5 \end{pmatrix}$ the algorithm returns the factor L printed

below (check: $LL^\top - A = 0$); for the indefinite matrix $\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$ it stops in the second step, where the radicand is $1 - 2^2 < 0$.

```

L =
[[ 2.      0.      0. ]
 [ 1.      3.      0. ]
 [-1.      1.      1.7321]]
L L^T - A =
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
indefinite matrix: matrix is not positive definite

```

2.6 Exploiting structure: banded and sparse matrices

Matrices from discretised differential equations are **sparse** — most entries are zero — and often **banded**: $a_{ij} = 0$ for $|i - j| > w$ (bandwidth w). Gaussian elimination without pivoting *preserves the band* (L and U have the same bandwidth), so the LU decomposition costs only $O(nw^2)$ instead of $\frac{2}{3}n^3$, and the substitutions $O(nw)$. For tridiagonal matrices ($w = 1$) this is the **Thomas algorithm** with $O(n)$ cost.

Example 2.13 (1D Poisson problem). The boundary value problem $-u''(t) = f(t)$ on $(0, 1)$, $u(0) = u(1) = 0$, discretised on the grid $t_i = ih$, $h = 1/(n + 1)$, with the central difference $-u''(t_i) \approx (-u_{i-1} + 2u_i - u_{i+1})/h^2$ gives the tridiagonal SPD system

$$\frac{1}{h^2} \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ & & -1 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ \vdots \\ u_n \end{pmatrix} = \begin{pmatrix} f(t_1) \\ \vdots \\ f(t_n) \end{pmatrix}.$$

Its eigenvalues are $\lambda_k = \frac{4}{h^2} \sin^2 \frac{k\pi h}{2}$, $k = 1, \dots, n$, so $\text{cond}_2 \approx \frac{4}{\pi^2 h^2} = O(n^2)$: finer grids give worse conditioning. This matrix will accompany us in Chapters 5, 6 and 10.

2.7 Summary

- $\text{cond}(A) = \|A\| \|A^{-1}\| \geq 1$ bounds the amplification of relative data errors in $Ax = b$; a small residual does not imply a small error unless $\text{cond}(A)$ is small.
- Gaussian elimination computes $PA = LU$ in $\frac{2}{3}n^3$ flops; solving for each right-hand side then costs $2n^2$. Partial pivoting keeps $|l_{ij}| \leq 1$ and makes the algorithm backward stable in practice — the error bound also contains the element growth factor, which is almost always small.
- SPD matrices admit the Cholesky decomposition $A = LL^T$ ($\frac{1}{3}n^3$ flops, no pivoting).
- Banded and sparse structure reduces the cost dramatically; direct solvers must preserve it, otherwise iterative methods (Chapter 6) are used.